STAGE UNIFUNL · RAPPORT TECHNIQUE

Conception et réalisation des outils web

Scraping de données, vérification de liens et analyse de contenu avec un modèle de langage

Stagiaire : [Nom et prénom à compléter]

Encadrant : [Nom à compléter]

Structure d'accueil : Unifunl

Période du stage : [Dates exactes à compléter]

Date du rapport : 24 octobre 2026

Résumé

Ce rapport décrit l'application web locale réalisée pour le projet de stage Unifunl. Elle regroupe un scraper spécialisé dans les pages de livres de démonstration, un vérificateur de liens cassés et une fonction facultative d'analyse de texte par modèle de langage. L'application Flask expose les fonctionnalités à travers des interfaces web et une API JSON. Un exemple Playwright traite également une page dont le contenu est généré en JavaScript.

La livraison inclut la validation des URL, la vérification des redirections, des limites de taille et de concurrence, des exports CSV protégés contre l'injection de formules, ainsi que des tests automatisés. L'analyse IA peut utiliser OpenAI ou Ollama exécuté localement. L'utilisateur a confirmé que l'analyse Ollama fonctionne sur son ordinateur; l'appel au service OpenAI n'a pas été réalisé.

64
tests automatisés réussis

60
produits récupérés sur 3 pages

2
outils web principaux

> **Portée du document.** Ce rapport technique décrit le logiciel et les vérifications effectuées. Il ne remplace pas le bilan personnel de stage. Les dates, les échanges avec le tuteur et les présentations doivent être complétés à partir des faits réels.

1. Objectifs et périmètre

Le projet vise à produire de petits outils web démontrant des opérations courantes de collecte et de vérification de données. L'application est conçue pour une démonstration locale. Elle ne doit pas être présentée comme un service public multi-utilisateur sécurisé sans renforcement complémentaire, notamment contre le DNS rebinding.

- Récupérer et structurer des informations de pages de livres sur Books to Scrape.
- Traiter la pagination et permettre le scraping parallèle d'URL indépendantes.
- Extraire des citations d'une page JavaScript avec Playwright.
- Repérer et classer les liens fonctionnels ou cassés d'une page.
- Fournir des interfaces, des routes JSON et des exports CSV.
- Analyser, sur demande explicite, le texte d'une page ou un texte saisi par l'utilisateur.

2. Architecture générale

Flask sert les deux interfaces et centralise les routes API. Les modules métier restent séparés : scraper statique, exemple Playwright, vérificateur de liens et client LLM. Les interfaces utilisent JavaScript pour appeler les routes et afficher les données. Le serveur de développement écoute sur `127.0.0.1:5000`.

Composant	Responsabilité	Technologies
Application web	Pages, validation des requêtes et API JSON	Flask, Jinja
Scraper de livres	Pages statiques, extraction, pagination, CSV/JSON	Requests, BeautifulSoup

Composant	Responsabilité	Technologies
Scraper dynamique	Chargement et extraction de citations après rendu JavaScript	Playwright, Chromium
Vérificateur de liens	Collecte et contrôle des liens HTTP(S), rapport CLI et interface	Requests, BeautifulSoup
Analyse LLM	Résumé, catégorie et extraction de champs structurés	OpenAI Responses API ou Ollama local
Interface	Saisie, états, résultats, affichage mobile et export	HTML, CSS, JavaScript

3. Fonctionnalités réalisées

3.1 Scraping de livres

Le scraper utilise Requests pour récupérer le HTML et BeautifulSoup pour en extraire le titre, le prix, la devise et l'image. Les champs manquants sont traités et les données peuvent être exportées en JSON et CSV. La pagination est bornée : trois pages par défaut et dix au maximum. L'ordre d'entrée des résultats est conservé.

La route `POST /api/scrape-batch` accepte de une à dix URL publiques indépendantes et limite la concurrence à trois téléchargements simultanés. La pagination suivie reste séquentielle, car l'URL suivante est découverte sur la page courante.

3.2 Scraping dynamique

Le script Playwright ouvre Quotes to Scrape, attend le rendu du contenu dynamique, puis extrait citations, auteurs et tags. Requests seul ne voit pas nécessairement les éléments insérés après le chargement de la page.

3.3 Vérificateur de liens

Le vérificateur extrait les liens absolus et relatifs, contrôle leur réponse HTTP et produit un rapport. Il est accessible en ligne de commande et depuis la page `/liens`. Le nombre maximal de liens est borné à 500.

3.4 Analyse de texte par IA

La fonction facultative retire les éléments non textuels de la page, limite le texte transmis à 12 000 caractères et demande une sortie conforme à un schéma JSON. Les champs retournés sont le résumé, la catégorie, le nom, la description et les caractéristiques. L'envoi n'a lieu qu'après action explicite de l'utilisateur.

- **OpenAI :** nécessite une clé API; l'application borne les appels à cinq par heure dans le processus courant.
- **Ollama :** exécute un modèle local à `127.0.0.1:11434`, sans clé OpenAI ni facturation API. L'utilisateur confirme que le mode local fonctionne.

4. Routes principales

Méthode et route	Fonction	Entrée principale
`GET /`	Interface scraper	—
`GET /liens`	Interface vérificateur	—

Méthode et route	Fonction	Entrée principale
`GET /api/health`	État de l'application	—
`GET /api/scrape?url=…`	Scraping paginé	URL
`POST /api/scrape-batch`	Scraping parallèle	`{"urls": […]}`
`POST /api/check-links`	Vérification de liens	`{"url": "…", "max_liens": 10}`
`POST /api/analyze-page`	Analyse du texte d'une page	`{"url": "https://…"}`
`POST /api/analyze-text`	Analyse de texte brut	`{"text": "…"}`

5. Technologies et organisation du dépôt

Fichier ou dossier	Rôle
`app.py`	Application Flask, validation et routes web/API
`scraper/first_scraper.py`	Scraper statique et exports
`scraper/playwright_scraper.py`	Exemple de scraping dynamique
`broken-links-checker/`	Vérificateur de liens en ligne de commande et tests
`broken_links_checker/`	Fonctions partagées de validation réseau
`templates/`, `static/`	Interfaces HTML, styles et scripts navigateur
`llm_analyzer.py`	Adaptateurs OpenAI/Ollama et validation JSON
`test_*.py`	Tests unitaires et tests d'API
`docs/`	Documentation, checklist et support de démonstration

6. Installation et démarrage

Environnement vérifié sous Python 3.13 sur Windows. Depuis PowerShell :

```
cd C:\Users\pc\Documents\stage-unifunl
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m playwright install chromium
python .\app.py
```

Ouvrir `http://127.0.0.1:5000` pour le scraper. La page de vérification est disponible à `http://127.0.0.1:5000/liens`.

Mode local Ollama

Installer Ollama pour Windows et télécharger le modèle, une seule fois :

```
ollama pull gemma3:4b
```

Dans le terminal qui lance Flask, sélectionner le fournisseur local :

```
$env:LLM_PROVIDER = "ollama"
$env:OLLAMA_MODEL = "gemma3:4b"
python .\app.py
```

Ollama doit être actif localement. Le modèle Gemma 3 4B téléchargé occupe environ 3,4 Go; la vitesse dépend du processeur et de la carte graphique. La procédure OpenAI et les précautions sur les clés sont dans le README.

7. Tests et résultats observés

Vérification	Résultat	Portée
Pytest	**64 réussis, 0 échec**	Inclut API Flask, validation, scraping, sorties CSV, limites, LLM et Ollama simulés
Compilation et dépendances	**Sans erreur; pip check propre**	Environnement virtuel du projet
Flask local	**HTTP 200 pour accueil, liens et santé**	Essai précédent de démarrage local
Books to Scrape	**60 produits sur trois pages**	Essai réseau du scraper parallèle
Quotes to Scrape	**10 citations extraites**	Essai réel via Playwright
Interface Chromium	**390 px et 1280 px, sans débordement horizontal**	Interactions navigateur simulées
Ollama	**Fonctionnement confirmé par l'utilisateur**	Pas relancé indépendamment dans l'environnement de cette vérification

Commande de vérification :

```
python -m compileall -q app.py scraper broken_links_checker broken-links-checker scripts
python -m pytest -v
python -m pip check
```

> **Évaluation LLM :** un jeu de trois exemples et un script d'évaluation sont fournis. Le score qualité n'a pas encore été produit. Avec Ollama, définir `LLM_PROVIDER=ollama` et `OLLAMA_MODEL=gemma3:4b`, puis lancer `python .\scripts\evaluer_llm.py`. Avec OpenAI, ces trois appels peuvent être facturés.

8. Sécurité, confidentialité et limites

- Les entrées URL acceptent HTTP(S); les hôtes locaux, identifiants intégrés et adresses IP non publiques sont rejetés.
- Les redirections sont suivies manuellement et chaque destination est validée avant la requête suivante.
- La taille de HTML téléchargé est limitée à 2 Mio. Les demandes JSON entrantes sont limitées à 64 Kio.

- Les appels parallèles sont limités à trois travailleurs; la liste d'URL de scraping parallèle est limitée à dix éléments.
- Le contenu HTML utilisateur est affiché comme texte; les cellules CSV susceptibles d'être interprétées comme formules sont neutralisées.
- Le contrôle DNS préalable ne neutralise pas complètement le DNS rebinding, car Requests résout le nom au moment de la connexion. Ne pas exposer l'outil comme service public multi-utilisateur sans correction supplémentaire.
- En mode Ollama, le texte est envoyé au service local. En mode OpenAI, le texte de page est transmis à OpenAI après le clic; ne transmettre que du contenu autorisé.
- Le quota OpenAI de cinq appels par heure est en mémoire et revient à zéro au redémarrage; configurer également une limite de dépenses chez le fournisseur.

9. Déploiement et tâches à finaliser

Aucun hébergeur n'est configuré et aucun déploiement public n'a été effectué. Render est proposé comme option de démonstration, avec Gunicorn, mais la mise en ligne exige le choix d'un compte, la vérification des limites de l'offre et l'autorisation de publication. Compte tenu de la limite DNS rebinding, le déploiement public ne doit pas être fait sans renforcement de la protection réseau.

Les éléments qui ne peuvent pas être déduits du code — dates exactes du stage, accueil, échanges avec le tuteur, retours de l'équipe, revue de code ou présentation formelle — doivent être confirmés et rédigés par la stagiaire. Les tâches 9, 10 et 14 peuvent aussi demander une clarification de l'encadrant : comparaison écrite Requests/Playwright, essais sur plusieurs structures de sites, et CSV compatible avec Excel plutôt qu'un fichier XLSX natif.

La liste complète des 30 tâches et de leurs preuves ou points à confirmer figure dans `docs/verification-taches-stage.md`. La trame de rapport de stage reste distincte de ce rapport technique.

Conclusion

Le dépôt fournit deux outils web fonctionnels, des scripts de scraping, des interfaces adaptatives, une API Flask, des exports, une analyse LLM à fournisseur sélectionnable et une suite de tests. Les vérifications automatisées et les essais réseau bornés attestent du fonctionnement local des composants décrits. Les étapes de validation personnelle, l'évaluation complète de qualité LLM et une éventuelle publication restent à terminer avant une remise définitive à l'encadrant.

Rapport technique du projet Stage Unifunl · 24 octobre 2026